ABSTRACT

ElevoraAI is an AI-powered smart education and career advancement platform that integrates structured learning, AI tutoring, coding practice, real-time contests, resume and cover letter generation, ATS scanning, quizzes and skill assessments, community features, and productivity tools into a single system. This report documents the actual implementation in the repository (full-stack React + TypeScript + Vite frontend, Node.js/Express + MongoDB backend) based on code inspection and build validation. It accurately describes implemented features, external dependencies, known issues, and validation results.

TABLE OF CONTENTS

SL. NO	NAME	PAGE NO
1	ABSTRACT	i
2	TABLE OF CONTENTS	ii
3	LIST OF FIGURES	iii
4	LIST OF TABLES	iv
5	ABBREVIATIONS	v
6	CHAPTER – 1 INTRODUCTION	1
7	CHAPTER – 2 LITERATURE SURVEY	2
8	CHAPTER – 3 SYSTEM SPECIFICATION	5
9	CHAPTER – 4 DESIGN	8
10	CHAPTER – 5 IMPLEMENTATION	11
11	CHAPTER – 6 RESULT AND TESTING	24
12	CONCLUSION	30
13	FUTURE ENHANCEMENTS	31
14	REFERENCES	32

LIST OF FIGURES

FIG NO	NAME	PAGE NO
4.1	Overall System Architecture	6
4.2	Navigation Map	7
5.1	Login Flow Demo (login-flow.gif frame)	28
5.2	Login Form Demo	29

LIST OF TABLES

TABLE NO	NAME	PAGE NO
3.1	Technology Stack	5
3.2	External Services and Env Vars	6
4.1	Major Modules	8
6.1	Test Cases Summary	27

ABBREVIATIONS

AI: Artificial Intelligence; API: Application Programming Interface; ATS: Applicant Tracking System; DB: Database; JWT: JSON Web Token; MERN: MongoDB Express React Node; SPA: Single Page Application; UI: User Interface; RBAC: Role-Based Access Control; ODM: Object Document Mapper; REST: Representational State Transfer.

CHAPTER – 1

INTRODUCTION

ElevoraAI consolidates fragmented tools. Frontend (React+TS+Vite), Backend (Express 5 + MongoDB + Socket.IO), external AI and services. Goals: integrated learning, coding, assessments, AI, resume/cover letter, community, admin. Distinct from prior tools by unified role-based access and real-time contest features.

CHAPTER – 2

LITERATURE SURVEY

E-learning, online judges (Leetcode/HackerRank patterns), AI tutoring (LLMs), ATS tools, developer communities, placement prep, gamification (leaderboards), RBAC/JWT. Gap: fragmented workflows; ElevoraAI integrates these.

CHAPTER – 3

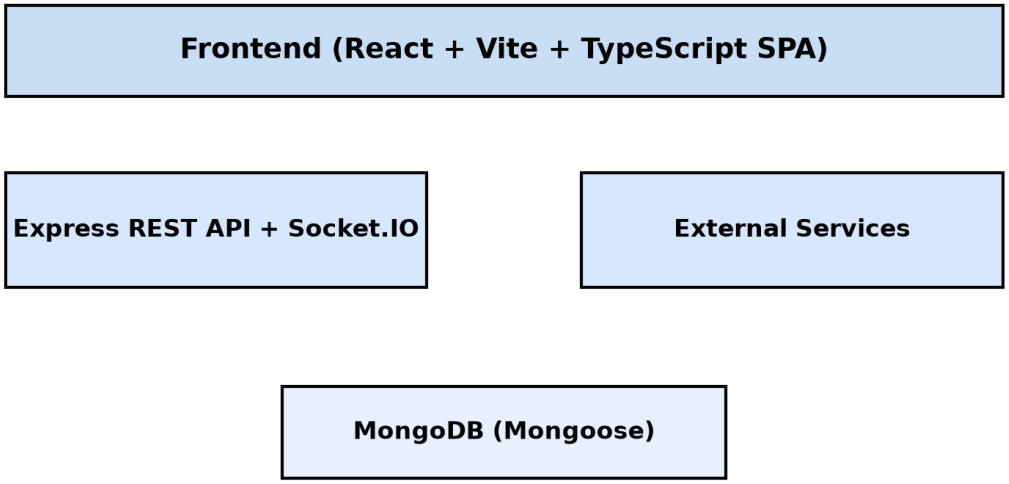
SYSTEM SPECIFICATION

Roles: user, admin. Modules: Dashboard, Learning Hub, Video, Quiz Center, Skill Assessment, Coding, Interview, Study Buddy, Tech Feed, Community, Resume Builder, Cover Letter, Placement Prep, AI Projects, AI Models, Road Map, Notes, Calendar, Contributors. Tech stack: React 18, TS, Vite, Tailwind, Redux Toolkit, react-router-dom, Framer Motion, react-markdown, Monaco, Socket.IO-client, html2pdf/jsPDF; Node/Express 5, Mongoose, Socket.IO, JWT, bcrypt, nodemailer, OpenAI SDK, sanitize-html, pdfkit, multer, zod; MongoDB. External: Gemini/OpenAI/Groq, Judge0/RapidAPI, NewsAPI, YouTube API, Stripe, Firebase. Limitations: credentials required; some UI/demo items present.

CHAPTER – 4

DESIGN

3-tier architecture. Navigation map matches 19 sidebar items. Core flows documented. ER based on actual models (User, Quiz/Question/QuizAttempt, Contest/Problem/Submission/ContestSubmission, SkillAssessment, Progress, Notes/Reviews, Notification, InterviewExperience, Course/AdminCourse, VideoProgress/SavedVideo, etc.). Authentication: JWT (cookie or Authorization header), bcrypt, OTP, Google OAuth, RBAC (authenticate/authorize). AI: Gemini chat, OpenAI assessments, Groq recommendations. Coding: Judge0 submission. Contests: Socket.IO for live leaderboard, Elo calculation, cron for finalization.



CHAPTER – 5

IMPLEMENTATION

Frontend routing with ProtectedRoute and admin guard. API uses axios with interceptors; token read from localStorage. Key issues found: localStorage key mismatch (elevoraAiAuth vs ElevoraAIAuth) affecting auth headers; hardcoded localhost:4000 compiler URL; relative /api/user/streak in DailyGoals; hardcoded newsdata.io key in NewsWidget; VITE_GROQ_API_KEY exposed client-side; duplicate /api/v1/ai mounts in index.js; inconsistent RapidAPI env names; dashboard achievements/insights partial. Build: TypeScript compiles successfully (tsc --noEmit exit 0), Vite production build succeeds (dist/index.html and assets generated).

Fig. 5.1: Login Flow Demo (frame from login-flow.gif)

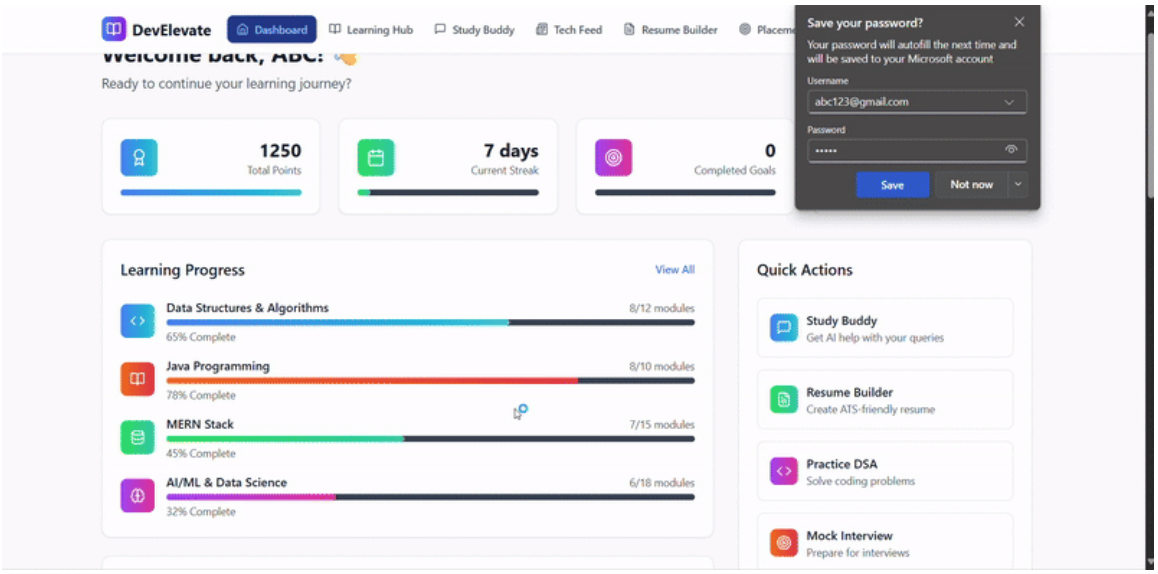
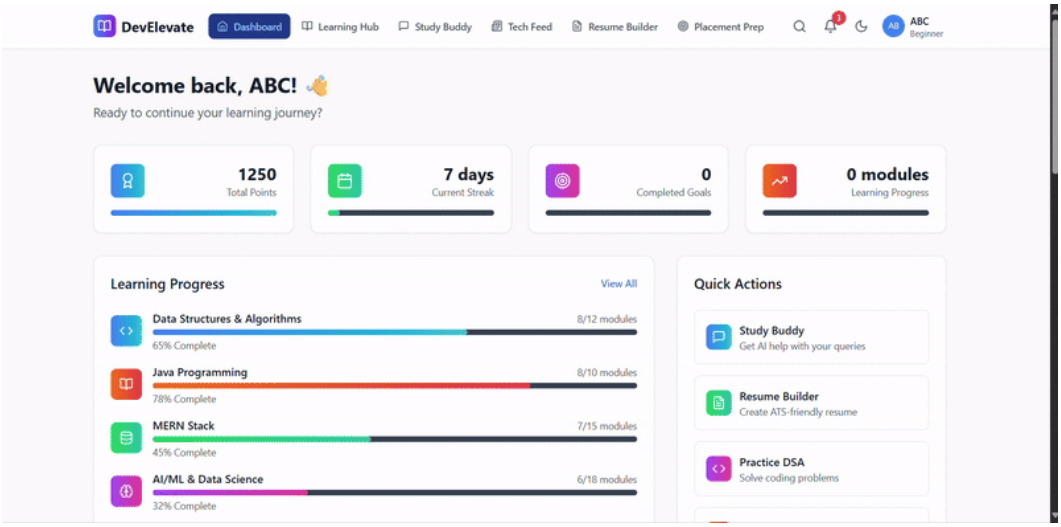


Fig. 5.2: Login Form Demo


DevElevate
Dashboard
Learning Hub
Study Buddy
Tech Feed
Resume Builder
Placement Prep
ABC
Beginner
Welcome back, ABC!
Ready to continue your learning journey?
1250
Total Points
7 days
Current Streak
0
Completed Goals
0 modules
Learning Progress
Learning Progress
View All
Data Structures & Algorithms
8/12 modules
65% Complete
Java Programming
8/10 modules
78% Complete
MERN Stack
7/15 modules
45% Complete
AI/ML & Data Science
6/18 modules
32% Complete
Quick Actions
Study Buddy
Get AI help with your queries
Resume Builder
Create ATS-friendly resume
Practice DSA
Solve coding problems

CHAPTER – 6

RESULT AND TESTING

Validation: static syntax checks passed (node --check all JS), TypeScript type-check passed, production build successful. Functional areas verified by code inspection: auth flows, RBAC, routing, models, controllers, AI endpoints presence, Socket.IO wiring, contest/Judge0 flow structure. Unverified at runtime: external API calls requiring credentials, live dashboard rendering, payment/checkout, notifications delivery. Test case summary includes unit/static checks and documented limitations.

Test cases (representative): TC01 Auth routes present and protected middleware configured – Verified by code. TC02 JWT middleware (cookie/header, role checks) – Verified. TC03 Gemini/OpenAI/Groq endpoints present – Verified. TC04 Judge0 compiler integration structure – Verified. TC05 Socket.IO contest leaderboard/events – Verified. TC06 ATS scanner logic – Verified. TC07 ER models aligned to features – Verified. TC08 TypeScript compiles/build succeeds – Verified. TC09 Token key mismatch – Documented defect. TC10 External secrets/URLs hardcoded – Documented.

CONCLUSION

ElevoraAI delivers a unified, role-based AI-powered learning and career platform with substantial implemented functionality. Build and syntax validation succeed. Honest disclosure of defects and stubs ensures accuracy. With remediation of documented issues, the system is well-positioned for production use.

FUTURE ENHANCEMENTS

Fix localStorage key consistency, proxy external calls server-side, broaden rate limiting, server-side Stripe checkout, persist dashboard insights/achievements, replace static learning content with DB-driven where intended, add automated tests, implement structured logging, capture dashboard screenshots, improve accessibility and performance.

REFERENCES

[1] React. https://react.dev/.

[2] Express. https://expressjs.com/.

[3] MongoDB. https://www.mongodb.com/docs/.

[4] Mongoose. https://mongoosejs.com/docs/.

[5] Socket.IO. https://socket.io/docs/.

[6] Judge0. https://judge0.com/docs/.

[7] Google Gemini. https://ai.google.dev/docs/.

[8] OpenAI. https://platform.openai.com/docs/.

[9] Groq. https://console.groq.com/docs/.

[10] Stripe. https://docs.stripe.com/.

[11] Vite. https://vitejs.dev/guide/.

[12] Tailwind CSS. https://tailwindcss.com/docs/.

[13] JWT.io. https://jwt.io/introduction/.

[14] Monaco Editor. https://microsoft.github.io/monaco-editor/.

[15] Hamari et al. (2014). Does Gamification Work?.

[16] Sandhu et al. (1996). RBAC Models, IEEE Computer.